

Module 11 Lab:
UML Class Diagram to Java Code Generation
Using Papyrus Software Designer

JR Clark (002035589)
NEU CSYE6255 (Summer 2026)
Professor Omar Badreldin / TA Karthik Kashyap

July 19, 2026

PART 1: Introduction and Background

This week's lab consists of a model-driven engineering workflow that is fully reproducible on current tooling: build a UML class diagram, assign attributes and typed relationships, generate Java source from that model, inspect and correct the generated output, and connect the exercise back to software testing concepts, including: model validation, design-to-code conformance, and unit/integration test identification.

The original lab assigned in Northeastern University's Canvas LMS could not be implemented as it required Papyrus for Real Time 1.0 (Papyrus-RT) which was archived (the final release was July 2017) (Eclipse Foundation, n.d.). Instead of the UML-RT PingPong model, the assignment was revised to use the actively maintained Papyrus Software Designer extension, which generates Java (as well as C, C++, and Python) from standard UML class diagrams.

The revised lab models a simplified Student Management System with five classes: Student, Course, Instructor, Department, and Enrollment. This domain was chosen specifically because it exercises the modeling concepts the lab is testing for with minimal extraneous complexity: primitive attributes (int, String), class-typed attributes that represent object references (e.g., department : Department), and one explicit linking class (Enrollment) that ties two other classes together. It is complex enough to require deliberate relationship modeling, but small enough that the generated code can be fully inspected by hand.

The Student Management System isn't a novel domain for MSIS students and it appears to be a deliberate callback to the INFO5100 introductory Java modeling exercise, letting the lab isolate the new skill (UML to codegen via Papyrus) rather than also teaching a new domain with novel example material.

PART 2: Installation and Environment

Environment: this lab was completed on a Mac running Apple Silicon (ARM64), with an existing Java 25 SDK and Eclipse installation already present on the machine. Eclipse Marketplace was used to locate the **Papyrus Desktop** which was incorrectly installed instead of Software Designer 3.2.0 (Figure 1, Figure 2) (Radermacher, n.d.). A dedicated Papyrus workspace was created consistent with the lab guidance (Figure 3).

Generating code against the model failed with a dependency-resolution error (see Appendix 1 for summary). **Modelio** was attempted next as a possible substitute, and was abandoned after four sequential, independently-diagnosed failures on macOS (Appendix 2). The Student Management model was then rebuilt and successfully carried through to generated Java in **Visual Paradigm Community Edition** (later switched to a Standard evaluation license, since CE does not include code generation), giving a complete, verified deliverable (Appendix 3)(Visual Paradigm, n.d.).

The root cause of the Papyrus failures turned out to be the installation of the incorrect download (see Appendix 4, Figure 39). With the correct tool installed and running, the model created in the earlier attempted opened directly inside it. Code generation, inspection, and verification were completed successfully using Papyrus SW Designer and IntelliJ IDEA (Appendix 4).

PART 3: UML Model Design

The five classes required by the lab were selected because together they exercise every modeling concept the exercise targets while keeping the domain small enough to inspect the generated code by hand: numeric identifiers, String attributes, class-typed object references, and one explicit linking class that ties two other classes together.

Student represents an individual enrolled in the institution and carries its own identity and academic affiliation. *Course* represents a single offered course record, tied to the instructor teaching it and the department offering it. *Instructor* represents a member of teaching staff. *Department* represents an academic administrative unit that includes both students and courses. *Enrollment* is the linking class: rather than attaching students to courses directly, it models a specific registration event by capturing an enrollment date and a grade and realizes the Student–Course relationship.

Each class was configured with the exact attribute set the lab specifies (see Appendix 1, Figures 7-12), which are either integer, string or references to the other classes. Primitive attributes (int, String) map directly onto Java's own primitive and standard library types. Class-typed attributes (ie, department: Department, instructor: Instructor, student: Student, course: Course), allow the generator to produce an actual object reference (private Department department;) instead of, for example, a String holding a department's name.

Model vs Diagram: the diagram and the model are not the same artifact. The diagram is a visual rendering of a subset of the model's elements for layout and communication. The model is the underlying semantic element tree, visible in Model Explorer, that the code generator actually reads when producing Java source.

This distinction matters directly for this lab: a property's diagram label can visually display the correct "name : Type" text even when its underlying Type field is unset, because the label is rendered from the Name field's text rather than derived from a resolved Type reference.

PART 4: Relationships and Model Configuration

The model encodes five logical relationships: Student–Department (a student is associated with a department), Course–Instructor (a course is taught by an instructor), Course–

Department (a course belongs to a department), Enrollment–Student (an enrollment references a student), and Enrollment–Course (an enrollment references a course).

All five relationships were represented using typed properties by giving the dependent class a property whose Type is the other class. For example, Student.department is a property named department with its Type set to the Department class, not a String containing the text "Department." This is functionally equivalent to a private field reference once generated into Java: Student.department : Department becomes private Department department; in the generated source.

Generalization was deliberately not used for any of these five relationships to avoid incorrect use. Generalization encodes an is-a relationship, for example, GraduateStudent is a Student would be a correct use of Generalization. None of the relationships in our model are that type of relationship and would have generated the wrong Java construct entirely (ie, an inheritance relationship, like Student extends Department, instead of a field reference).

Entering a property's Name and Type in separate fields, rather than typing both into the Name field as a single string, was treated as a hard rule throughout this lab rather than a formatting preference. The code generator reads the property's underlying semantic Type slot, not the text rendered in the diagram. Typing department : Department entirely into the Name field, with Type left at <no value>, can look identical to a correctly-configured property in the diagram view while having no assigned type underneath and would result an invalid placeholder field. Every property in the final Papyrus model was verified in Model Explorer to confirm Name and Type were populated as two distinct fields (Figures 41 & 42).

PART 5: Java Code Generation and Error Resolution

With Papyrus SW Designer correctly installed, code generation workflow was carried out on the completed five-class Student Management model (Figure 42, Figure 43). The generation workflow prompted creation of the target JDT project (Figure 44) as specified in the lab instructions. This produced a package containing all five required files and alongside their compiled .class outputs (Figure 45), matching the required one UML class to one Java file mapping exactly.

Each generated file was inspected against the model and against the lab document's own expected Java. Figures 33-38 (VP), and 50-54 (Papayrus SW Designer) opened side by side with the lab's expected output, matched it exactly, field for field, for every single class.

The lab specifically calls out an undef defect that can occur when a property's Type is left unassigned, producing an invalid declaration such as `public undef email : String;` instead of a valid `private String email;`. That failure mode was avoided in this final generation pass because every property across all five classes had already been verified in Model Explorer to have Name and Type set as two separate fields. No undef placeholders, unresolved types, or manual corrections were needed in the generated output; the only correction made anywhere in this process was an earlier casing fix (`studentID` to `studentId`) applied to the Student identifier property to match the lab's required naming exactly.

PART 6: Testing Relevance, Results, and Conclusion

Model validation was performed before generation by reviewing every property of all five classes in Model Explorer and confirming none displayed <no value> in place of an assigned Type. *Design-to-code conformance* was verified by comparing each generated Java file directly against the lab document's expected output, matching exactly (Figures 33-38 and 50-54). *Compilation verification* was carried out by opening the generated project in IntelliJ IDEA with a

clean result and no errors (Figure 49), confirming the generated project is not only structurally present but compiles as valid, resolvable Java.

The class diagram identifies five natural *unit-test targets*, one per class: StudentTest, CourseTest, InstructorTest, DepartmentTest, and EnrollmentTest. The five typed relationships identify the corresponding integration boundaries that must be tested together: Student–Department, Course–Instructor, Course–Department, Enrollment–Student, and Enrollment–Course (see Appendix 5).

This lab's most direct lesson was that a UML model can look entirely correct in its diagram while being incomplete underneath. The generated code is the real source of truth for validating configuration. Second, the same Papyrus Software Designer release can fail or succeed depending on the installation path, not just the version. Diagnosing that distinction, rather than assuming the tool was simply broken, turned out to be as valuable a piece of technical troubleshooting as the modeling exercise itself.

This exercise is intentionally narrower than the original Papyrus-RT lab it replaced. Everything demonstrated here is structural: a static class model transformed into static Java source and verified by inspection and compilation, not by observing runtime behavior.

In summary, the Student Management System was modeled with all five required classes, their required attributes, and their required typed references; the model was validated before generation; Java code was generated directly in Papyrus Software Designer and inspected against both the model and the lab's expected output field by field; the installation obstacles that stood in the way were diagnosed to a specific, well-evidenced root cause rather than worked around indefinitely; and the resulting project compiles cleanly, closing the loop from UML design to verified Java implementation using the actual assigned tool.

APPENDIX 1: Evidence Technical Implementation for Papyrus Desktop

Papyrus Desktop would not generate code because the Papyrus Designer Components feature was missing and could not be installed because its core model library requires org.eclipse.papyrus.marte.static.profile 1.2.2, a MARTE profile bundle that was not present on the official 3.2.0 update site. To determine whether this was specific to the 3.2.0 release, the same Designer Components feature was tested against two older Papyrus Software Designer releases. Both failed with the identical missing-requirement error against org.eclipse.papyrus.marte.static.profile 1.2.2, and p2's own Install Remediation Page confirmed the same dependency was blocking each version independently. Other classmates reported installation errors as well which inspired me to explore alternatives for generating Java code from UML models.

See Claude diagnostic discussion here: <https://claude.ai/share/6d74f648-30c2-446f-8ff7-49579775cfda>

Figure 1. Java JDK verification



```
blackcatranch-m3 — -zsh — 80x24

[blackcatranch-m3@JRC-M3 ~ % java -version
openjdk version "25.0.3" 2026-04-21 LTS
OpenJDK Runtime Environment Temurin-25.0.3+9 (build 25.0.3+9-LTS)
OpenJDK 64-Bit Server VM Temurin-25.0.3+9 (build 25.0.3+9-LTS, mixed mode, sharing)
blackcatranch-m3@JRC-M3 ~ % ]
```

Figure 2. Download Papyrus from Eclipse.org & Install

All downloads are provided under the terms and conditions of the [Eclipse Foundation Software User Agreement](#) unless otherwise specified.

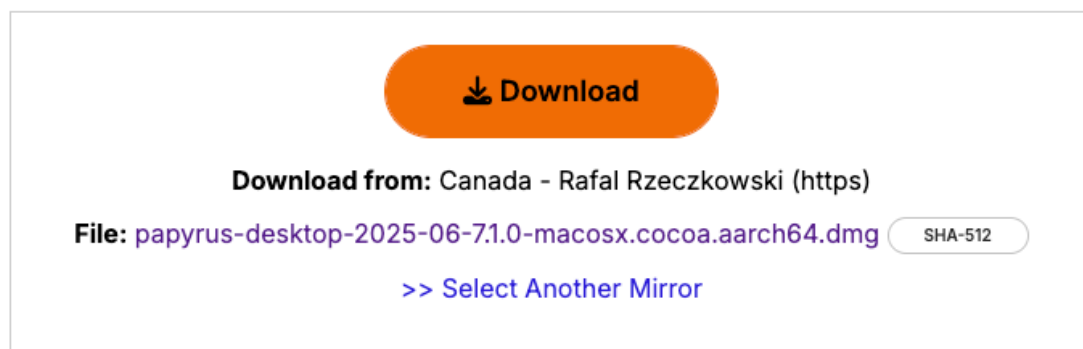


Figure 3. Workspace for Papyrus

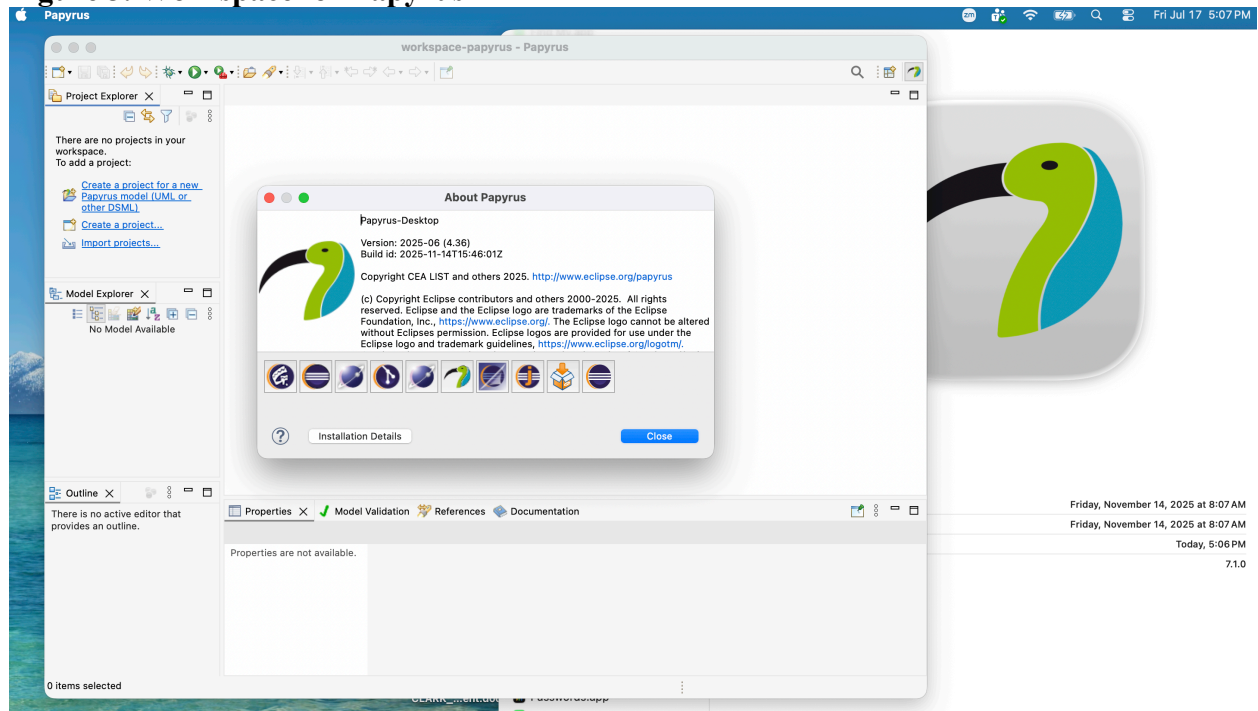


Figure 4. New project creation in Papyrus

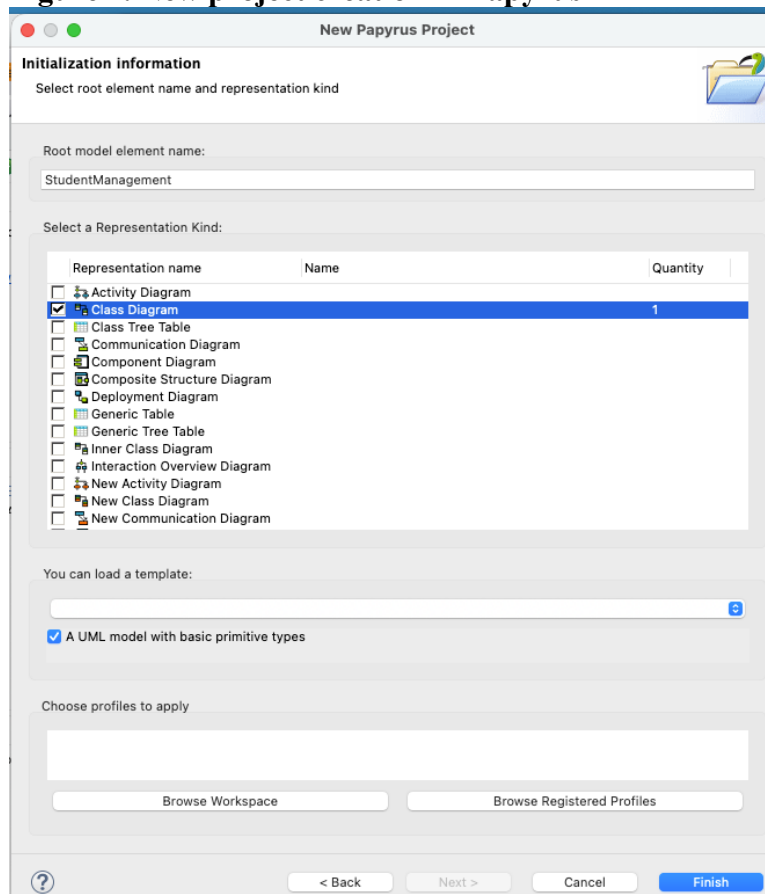


Figure 5. Empty class diagram in Papyrus

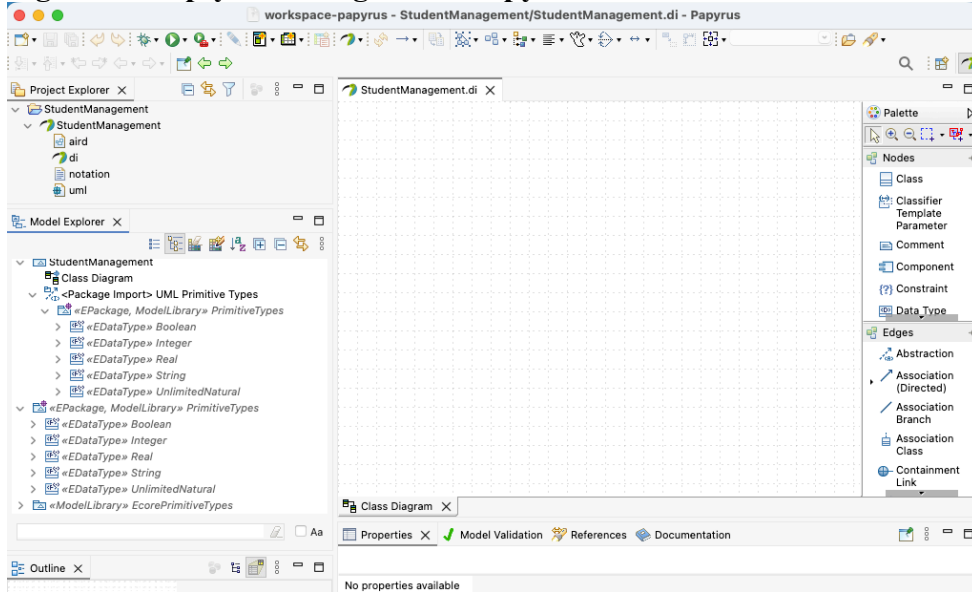


Figure 6. Five created classes.

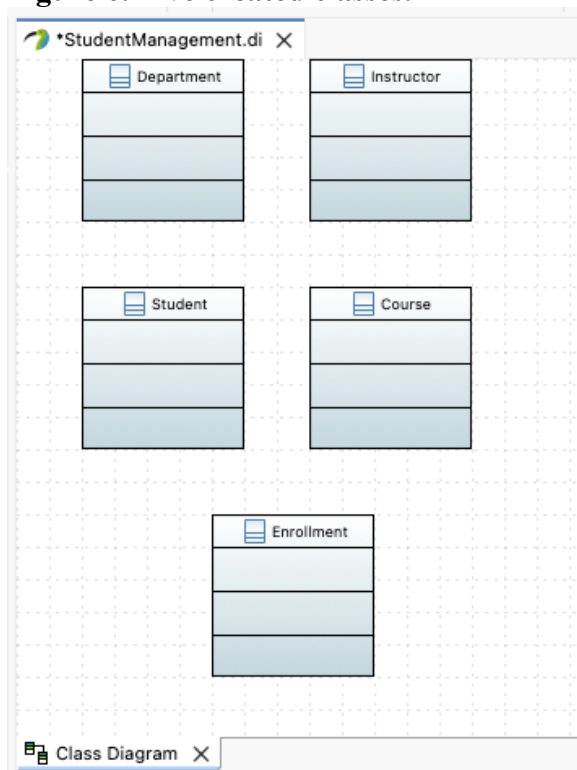


Figure 7. Student class properties.

 Student

UML	Name		Student
Comments	Label		
Profile			
Advanced	Qualified name		StudentManagement::Student
			☐ Is abstract
			☐ Is active
	Visibility		public 
	Owned attribute		 studentId : Integer  name : String  email : String  department : Department

Figure 8. Course class properties.

 Course

UML	Name		Course
Comments	Label		
Profile			
Advanced	Qualified name		StudentManagement::Course
			☐ Is abstract
			☐ Is active
	Visibility		public 
	Owned attribute		 courseId : Integer  courseName : String  credits : Integer  instructor : Instructor  department : Department

Figure 9. Instructor class properties.

 Instructor

UML	Name		Instructor
Comments	Label		
Profile			
Advanced	Qualified name		StudentManagement::Instruct
			☐ Is abstract
			☐ Is active
	Visibility		public 
	Owned attribute		 instructorId : Integer  name : String  email : String

Figure 10. Department class properties.

The screenshot shows the 'Department' class properties dialog box. It has a sidebar with tabs: UML, Comments, Profile, and Advanced. The 'UML' tab is selected. The main area contains the following fields:

- Name: Department
- Label: (empty)
- Qualified name: StudentManagement::Department
- Is abstract: ☐
- Is active: ☐
- Visibility: public
- Owned attribute: departmentId : Integer, departmentName : String

Figure 11. Enrollment class properties.

The screenshot shows the 'Enrollment' class properties dialog box. It has a sidebar with tabs: UML, Comments, Profile, and Advanced. The 'UML' tab is selected. The main area contains the following fields:

- Name: Enrollment
- Label: (empty)
- Qualified name: StudentManagement::Enrollment
- Is abstract: ☐
- Is active: ☐
- Visibility: public
- Owned attribute: enrollmentId : Integer, enrollmentDate : String, grade : String, student : Student, course : Course

Figure 12. Model Explorer showing all model elements.

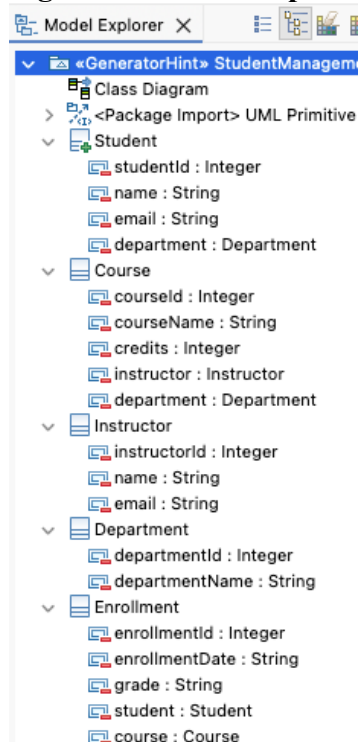


Figure 13. Example Enrollment / Student properties view showing a separate Name and Type

 **student : Student**

UML	Name	?	student
Comments	Label	?	
Profile		?	☐ Is derived
Advanced		?	☐ Is derived union
		?	☐ Is ordered
		?	☐ Is read only
		?	☐ Is static
		?	☒ Is unique
	Visibility	?	private 
	Aggregation	?	none 

APPENDIX 2: Modelio installation

Modelio was the first alternative attempted when the MARTE issue was encountered with the (wrong version of) Papyrus. Modelio was abandoned after four sequential, independently-diagnosed failures on macOS: a Gatekeeper security block on first launch, a JVM/application architecture mismatch, a missing JAXB module under JDK 11 (Modelio's required runtime), and a native SWT rendering fault after the JDK was downgraded.

See Claude diagnostic discussion here: <https://claude.ai/share/6d74f648-30c2-446f-8ff7-49579775cfd4>

Figure 14. Brew installation of Modelio

```
blackcatranch-m3 --zsh -- 168x70

==> Would install 1 cask:
modelio
==> Fetching downloads for: modelio
Warning: modelio has been deprecated because it does not pass the macOS Gatekeeper check! It will be disabled on 2026-09-01.
✓ Cask modelio (4.1.0) Downloaded 200.0MB/200.0MB
Warning: modelio has been deprecated because it does not pass the macOS Gatekeeper check! It will be disabled on 2026-09-01.
==> Caveats
modelio is built for Intel macOS and so requires Rosetta 2 to be installed.
You can install Rosetta 2 with:
  softwareupdate --install-rosetta --agree-to-license
Note that it is very difficult to remove Rosetta 2 once it is installed.

==> Installing Cask modelio
==> Moving App 'Modelio 4.1.app' to '/Applications/Modelio 4.1.app'
modelio was successfully installed!
==> 'brew cleanup' has not been run in the last 30 days, running now...
Disable this behaviour by setting 'HOMEBREW_NO_INSTALL_CLEANUP=1'.
Hide these hints with 'HOMEBREW_NO_ENV_HINTS=1' (see 'man brew').
Removing: /Users/blackcatranch-m3/Library/Caches/Homebrew/aria2--1.37.0_1... (14.1KB)
Removing: /Users/blackcatranch-m3/Library/Caches/Homebrew/aria2--1.37.0_1... (1.7MB)
Removing: /Users/blackcatranch-m3/Library/Caches/Homebrew/cabextract_bottle_manifest--1.11... (12.3KB)
Removing: /Users/blackcatranch-m3/Library/Caches/Homebrew/cabextract--1.11... (59.2KB)
Removing: /Users/blackcatranch-m3/Library/Caches/Homebrew/cdrtools_bottle_manifest--3.02a09... (14.8KB)
Removing: /Users/blackcatranch-m3/Library/Caches/Homebrew/cdrtools--3.02a09... (2.0MB)
Removing: /opt/homebrew/Cellar/graphite2/1.3.14... (19 files, 278.2KB)
Removing: /Users/blackcatranch-m3/Library/Caches/Homebrew/graphite2_bottle_manifest--1.3.14-2... (8.0KB)
Removing: /Users/blackcatranch-m3/Library/Caches/Homebrew/graphite2--1.3.14... (92.5KB)
Removing: /opt/homebrew/Cellar/harfbuzz/14.2.0... (94 files, 13.2MB)
Removing: /Users/blackcatranch-m3/Library/Caches/Homebrew/harfbuzz_bottle_manifest--14.2.0... (34.2KB)
Removing: /Users/blackcatranch-m3/Library/Caches/Homebrew/harfbuzz--14.2.0... (3.5MB)
Removing: /Users/blackcatranch-m3/Library/Caches/Homebrew/libssh2_bottle_manifest--1.11.1... (11.8KB)
Removing: /Users/blackcatranch-m3/Library/Caches/Homebrew/libssh2--1.11.1... (423.8KB)
Removing: /Users/blackcatranch-m3/Library/Caches/Homebrew/readline_bottle_manifest--8.3.3... (10.0KB)
Removing: /Users/blackcatranch-m3/Library/Caches/Homebrew/readline--8.3.3... (762KB)
Removing: /Users/blackcatranch-m3/Library/Caches/Homebrew/sqlite_bottle_manifest--3.51.2... (11.4KB)
Removing: /Users/blackcatranch-m3/Library/Caches/Homebrew/sqlite--3.51.2... (2.4MB)
Removing: /Users/blackcatranch-m3/Library/Caches/Homebrew/wimlib_bottle_manifest--1.14.4-1... (12.2KB)
Removing: /Users/blackcatranch-m3/Library/Caches/Homebrew/wimlib--1.14.4... (573.6KB)
Removing: /Users/blackcatranch-m3/Library/Caches/Homebrew/chntpw--0.99.6.tar.gz... (95.9KB)
Removing: /Users/blackcatranch-m3/Library/Caches/Homebrew/portable-ruby-4.0.5_1.arm64_big_sur.bottle.tar.gz... (12.7MB)
Removing: /Users/blackcatranch-m3/Library/Caches/Homebrew/openssl@1.0--patch--4ad22bcfc85171a25f035b6fc47c7140752b9ed7467bb560... (5.4MB)
Removing: /Users/blackcatranch-m3/Library/Caches/Homebrew/openssl@1.0--1.0.2u.tar.gz... (5.4MB)
Removing: /Users/blackcatranch-m3/Library/Caches/Homebrew/bootsnap/b652ffca4415f01425a6d96d80cd16a3089b004d15edd0b0b2523e68e8d... (5.4MB)
Removing: /Users/blackcatranch-m3/Library/Caches/Homebrew/bootsnap/ef2555df5dcbaacf64096a0cf3397f7e94ff38d2dfe5dacc95041f02350... (5.4MB)
Removing: /Users/blackcatranch-m3/Library/Caches/Homebrew/bootsnap/16e4b77f396cb05f9c9d7cc3db325ed737f1a41f52ffbca011c77ad70f5f... (5.4MB)
Removing: /Users/blackcatranch-m3/Library/Logs/Homebrew/libtiff... (64B)
```

Figure 15. Bypassing the security issue for Modelio

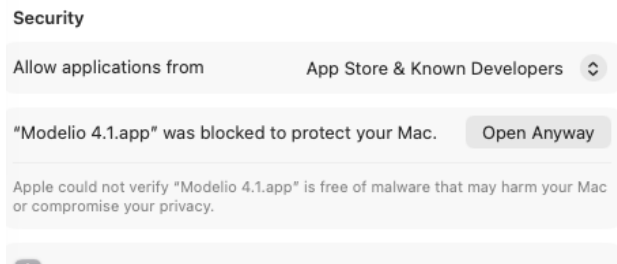


Figure 16. Noted Java Mismatch

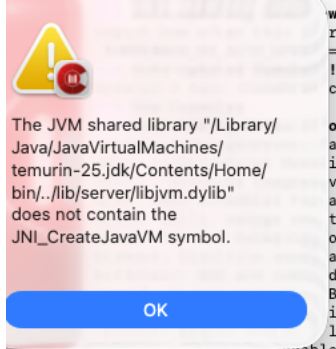


Figure 17. Installing older version of Java & all JDK locations:

```
Last login: Fri Jul 17 16:46:41 on ttys000
[blackcatranch-m3@JRC-M3 ~ % brew install --cask temurin@11
✓ JSON API packages.arm64_tahoe.jws.json           Downloaded 15.5MB/ 15.5MB
==> Would install 1 cask:
temurin@11
==> Fetching downloads for: temurin@11
✓ Cask temurin@11 (11.0.31,11)                       Downloaded 185.0MB/185.0MB
==> Installing Cask temurin@11
==> Running installer for temurin@11 with `sudo` (which may request your password)
[Password:
installer: Package name is Eclipse Temurin
installer: Installing at base path /
installer: The install was successful.
📦 temurin@11 was successfully installed!
[blackcatranch-m3@JRC-M3 ~ % /usr/libexec/java_home -V
Matching Java Virtual Machines (3):
  25.0.3 (arm64) "Eclipse Adoptium" - "OpenJDK 25.0.3" /Library/Java/JavaVirtualMachines/temurin-25.jdk/Contents/Home
  19.0.1 (arm64) "Oracle Corporation" - "Java SE 19.0.1" /Library/Java/JavaVirtualMachines/jdk-19.jdk/Contents/Home
  11.0.31 (arm64) "Eclipse Adoptium" - "OpenJDK 11.0.31" /Library/Java/JavaVirtualMachines/temurin-11.jdk/Contents/Home
/Library/Java/JavaVirtualMachines/temurin-25.jdk/Contents/Home
blackcatranch-m3@JRC-M3 ~ %
```

Figure 18. Identify Modelio config file

```
[blackcatranch-m3@JRC-M3 ~ % find "/Applications/Modelio 4.1.app" -iname "*.ini"
/Applications/Modelio 4.1.app/Contents/Eclipse/configuration/config.ini
/Applications/Modelio 4.1.app/Contents/Eclipse/plugins/org.eclipse.platform_4.9.0.v20180906-0745/plugin_customization.ini
/Applications/Modelio 4.1.app/Contents/Eclipse/plugins/org.eclipse.platform_4.9.0.v20180906-0745/about.ini
/Applications/Modelio 4.1.app/Contents/Eclipse/modelio.ini
blackcatranch-m3@JRC-M3 ~ %
```

Figure 19. View contents of Modelio config file

```
[blackcatranch-m3@JRC-M3 ~ % cat "/Applications/Modelio 4.1.app/Contents/Eclipse/modelio.ini"
ni"
-startup
../Eclipse/plugins/org.eclipse.equinox.launcher_1.5.100.v20180827-1352.jar
--launcher.library
../Eclipse/plugins/org.eclipse.equinox.launcher.cocoa.macosx.x86_64_1.1.800.v20180827-135
2
-clearPersistedState
-vmargs
-Xms512m
-Dpython.console.encoding=UTF-8
-Dosgi.requiredJavaVersion=1.8
--add-modules=ALL-SYSTEM
-XstartOnFirstThread
-Dorg.eclipse.swt.internal.carbon.smallFonts
blackcatranch-m3@JRC-M3 ~ %
```

Figure 20. Install Eclipse Temurin 11

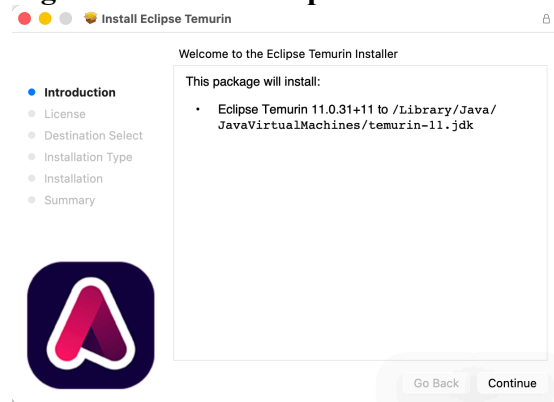


Figure 21. Update Modelio config.ini to point to JDK 11

```
blackcatranch-m3 — -zsh — 89x49

[blackcatranch-m3@JRC-M3 ~ % cat "/Applications/Modelio 4.1.app/Contents/Eclipse/modelio.ini"
ni"
-startup
../Eclipse/plugins/org.eclipse.equinox.launcher_1.5.100.v20180827-1352.jar
--launcher.library
../Eclipse/plugins/org.eclipse.equinox.launcher.cocoa.macosx.x86_64_1.1.800.v20180827-135
2
-clearPersistedState
-vm
/Library/Java/JavaVirtualMachines/temurin-11.jdk/Contents/Home/lib/server/libjvm.dylib
-vmargs
-Xms512m
-Dpython.console.encoding=UTF-8
-Dosgi.requiredJavaVersion=1.8
--add-modules=ALL-SYSTEM
-XstartOnFirstThread
-Dorg.eclipse.swt.internal.carbon.smallFonts
blackcatranch-m3@JRC-M3 ~ %
```

Figure 22. Adding another JDK (8) to the system:

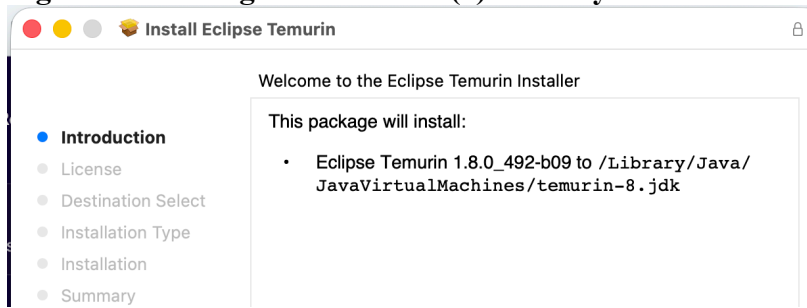


Figure 23. New error

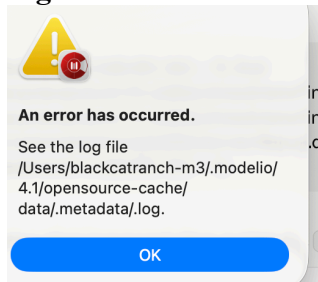
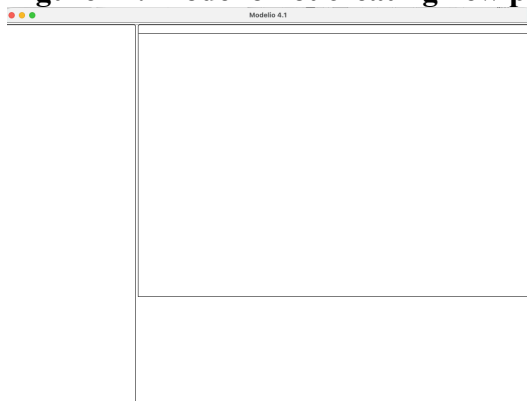


Figure 24. Modelio not creating new projects



APPENDIX 3: Visual Paradigm Community Edition

Visual Paradigm was the second alternative implemented to generate java code from a UML model. It was easy to install and at this point – creating the classes was easy to do manually.

This was recommended by Claude in the same discussion referenced in Appendices 1 and 2. See Claude diagnostic discussion here: <https://claude.ai/share/6d74f648-30c2-446f-8ff7-49579775cfda>

Figure 25. Visual Paradigm CE Desktop

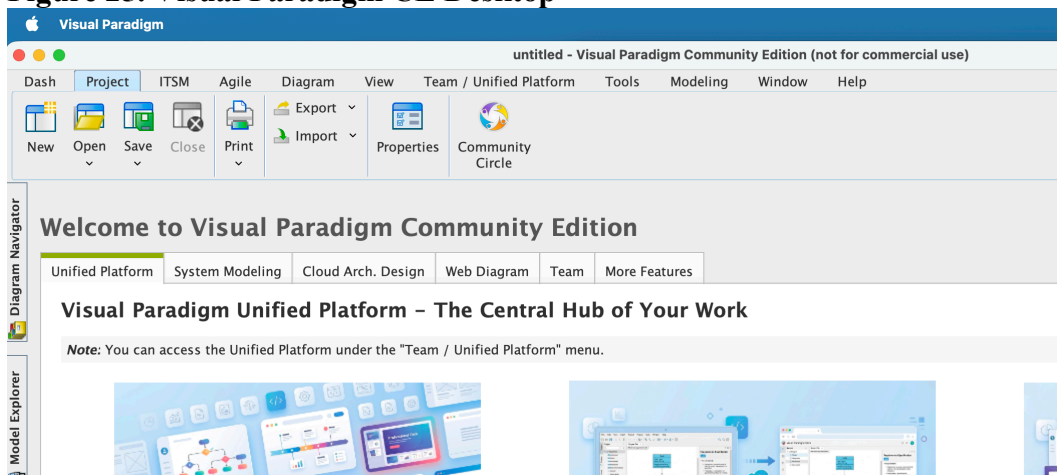


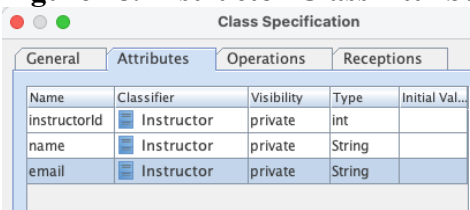
Figure 26. Student Class attributes

Class Specification				
General Attributes Operations Receptions				
Name	Classifier	Visibility	Type	Initial Value
studentID	Student	private	int	
name	Student	private	String	
email	Student	private	String	
department	Student	private	Department	

Figure 27. Course Class Attributes

Class Specification				
General Attributes Operations Receptions				
Name	Classifier	Visibility	Type	Initial Value
courseId	Course	private	int	
courseName	Course	private	String	
credits	Course	private	int	
instructor	Course	private	Instructor	
department	Course	private	Department	

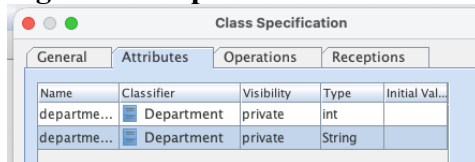
Figure 28. Instructor Class Attributes



The image shows a 'Class Specification' window with the 'Attributes' tab selected. It displays a table of attributes for the 'Instructor' class.

Name	Classifier	Visibility	Type	Initial Val...
instructorId	Instructor	private	int	
name	Instructor	private	String	
email	Instructor	private	String	

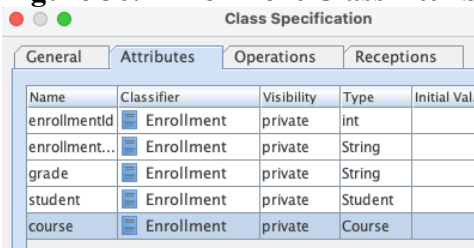
Figure 29. Department Class Attributes



The image shows a 'Class Specification' window with the 'Attributes' tab selected. It displays a table of attributes for the 'Department' class.

Name	Classifier	Visibility	Type	Initial Val...
departmentId	Department	private	int	
departmentName	Department	private	String	

Figure 30. Enrollment Class Attributes



The image shows a 'Class Specification' window with the 'Attributes' tab selected. It displays a table of attributes for the 'Enrollment' class.

Name	Classifier	Visibility	Type	Initial Val...
enrollmentId	Enrollment	private	int	
enrollmentDate	Enrollment	private	String	
grade	Enrollment	private	String	
student	Enrollment	private	Student	
course	Enrollment	private	Course	

Figure 31. Model Explorer View of Classes & Attributes

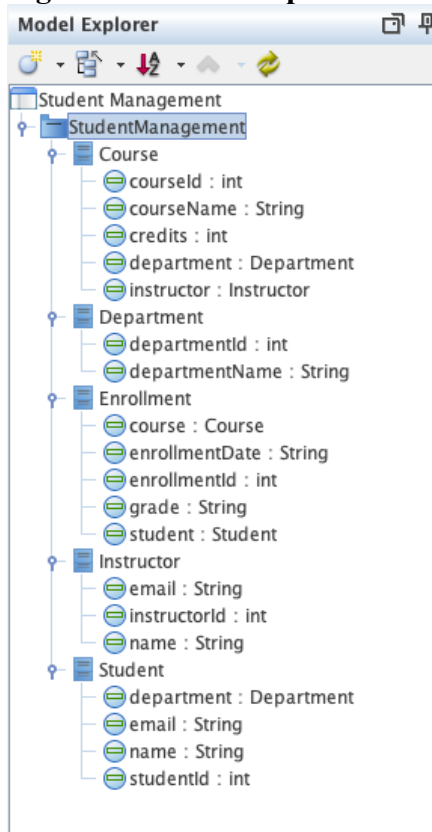


Figure 32. Files generated by VP

StudentManagement	Today at 9:47 PM
Course.java	Today at 9:47 PM
Department.java	Today at 9:47 PM
Enrollment.java	Today at 9:47 PM
Instructor.java	Today at 9:47 PM
Student.java	Today at 9:47 PM

Figure 33. Student.java validation

```
1 package StudentManagement;
2
3 public class Student { 1 usage
4
5     private int studentId; no usages
6     private String name; no usages
7     private String email; no usages
8     private Department department; no usages
9
10 }
```

Student.java

```
package StudentManagement;

public class Student {

    private int studentId;
    private String name;
    private String email;
    private Department department;
}
```

Figure 34. Course.java validation

```
1 package StudentManagement;
2
3 public class Course { 1 usage
4
5     private int courseId; no usages
6     private String courseName; no usages
7     private int credits; no usages
8     private Instructor instructor; no usages
9     private Department department; no usages
10
11 }
```

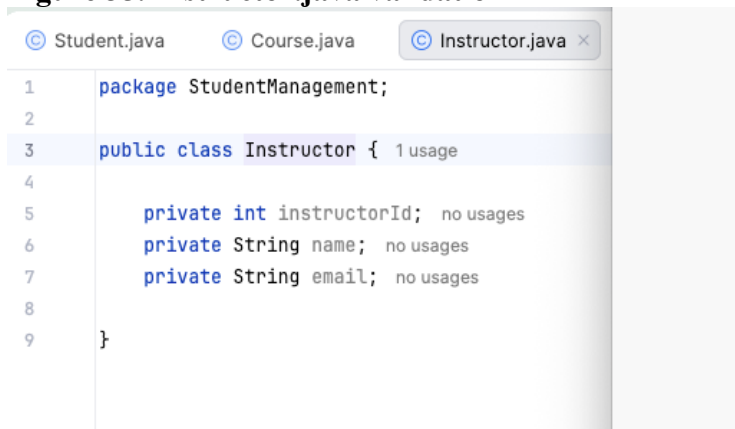
Course.java

```
package StudentManagement;

public class Course {

    private int courseId;
    private String courseName;
    private int credits;
    private Instructor instructor;
    private Department department;
}
```

Figure 35. Instructor.java validation



```
1 package StudentManagement;
2
3 public class Instructor { 1 usage
4
5     private int instructorId; no usages
6     private String name; no usages
7     private String email; no usages
8
9 }
```

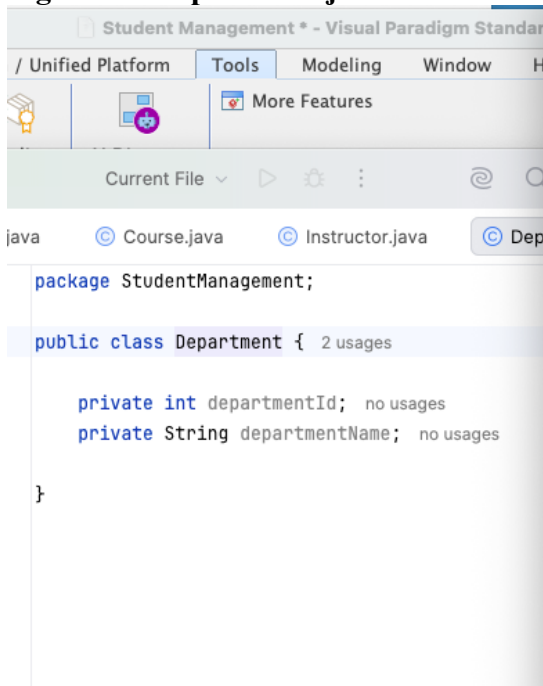
Instructor.java

```
package StudentManagement;

public class Instructor {

    private int instructorId;
    private String name;
    private String email;
}
```

Figure 36. Department.java validation



```
1 package StudentManagement;
2
3 public class Department { 2 usages
4
5     private int departmentId; no usages
6     private String departmentName; no usages
7
8 }
```

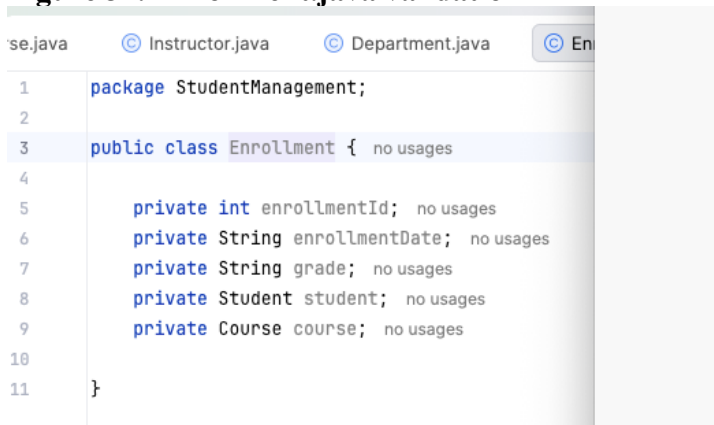
Department.java

```
package StudentManagement;

public class Department {

    private int departmentId;
    private String departmentName;
}
```

Figure 37. Enrollment.java validation



```
1 package StudentManagement;
2
3 public class Enrollment { no usages
4
5     private int enrollmentId; no usages
6     private String enrollmentDate; no usages
7     private String grade; no usages
8     private Student student; no usages
9     private Course course; no usages
10
11 }
```

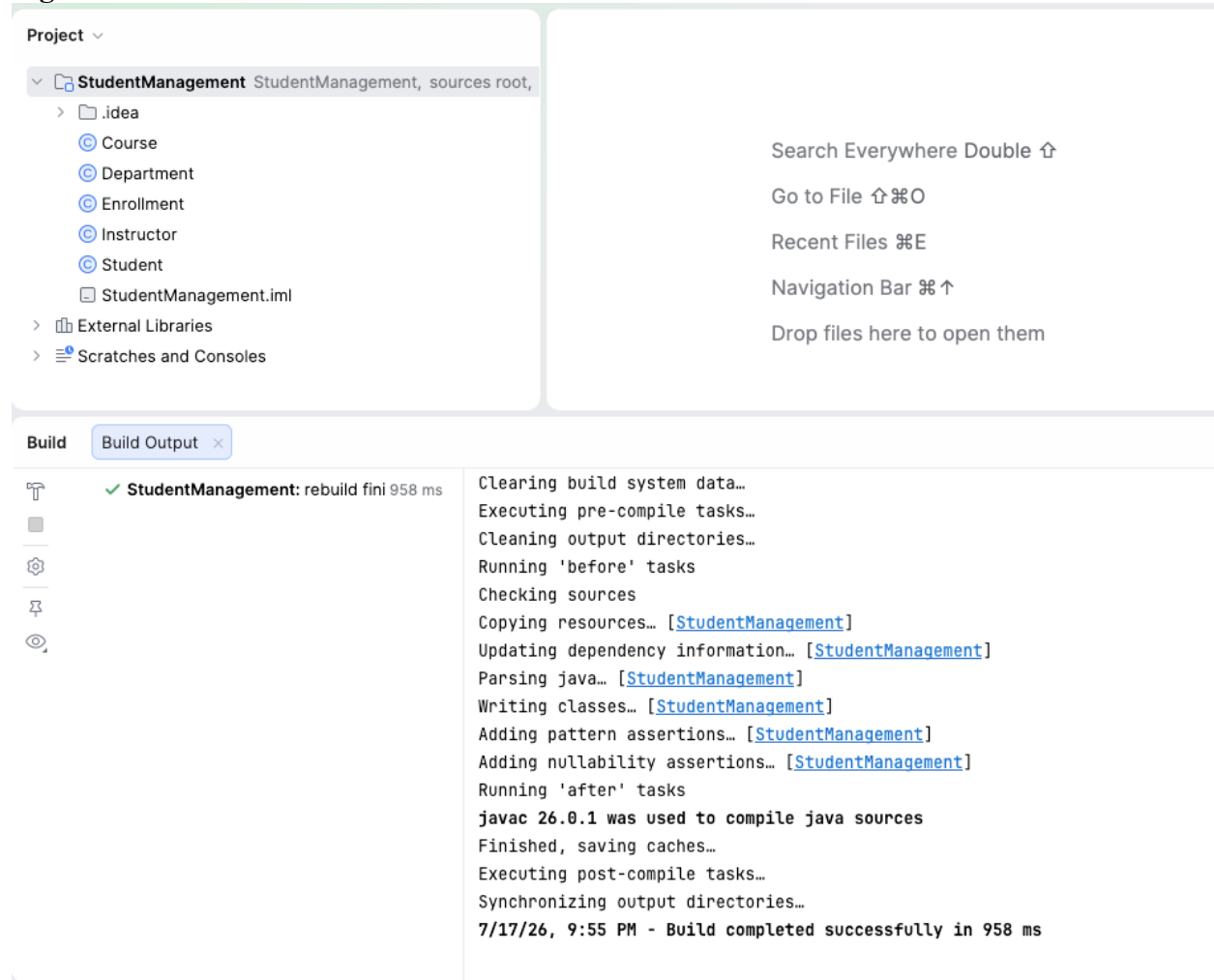
Enrollment.java

```
package StudentManagement;

public class Enrollment {

    private int enrollmentId;
    private String enrollmentDate;
    private String grade;
    private Student student;
    private Course course;
}
```

Figure 38. Results of IntelliJ IDEA build



APPENDIX 4: Download of the correct version of “Papyrus” -- Papyrus SW Designer 3.2.0

Figure 39. Screen capture of the Downloads folder showing

Name	Date Created
p2.index	Yesterday at 8:06 PM
papyrus-desktop-2025-06-7.1.0-macosx.cocoa.aarch64.dmg	Yesterday at 5:04 PM
papyrus-desktop-designer-2025-06-3.2.0-macosx.cocoa.aarch64.dmg	Today at 9:57 AM

Figure 40. Gatekeeper popup for Papyrus-designer

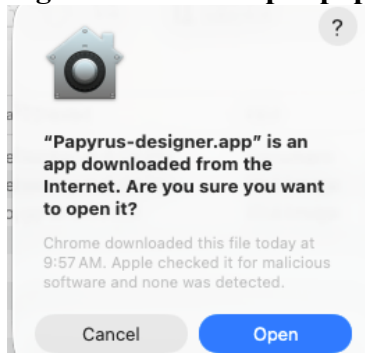


Figure 41. Papyrus SW Designer workspace with all 5 classes created imported

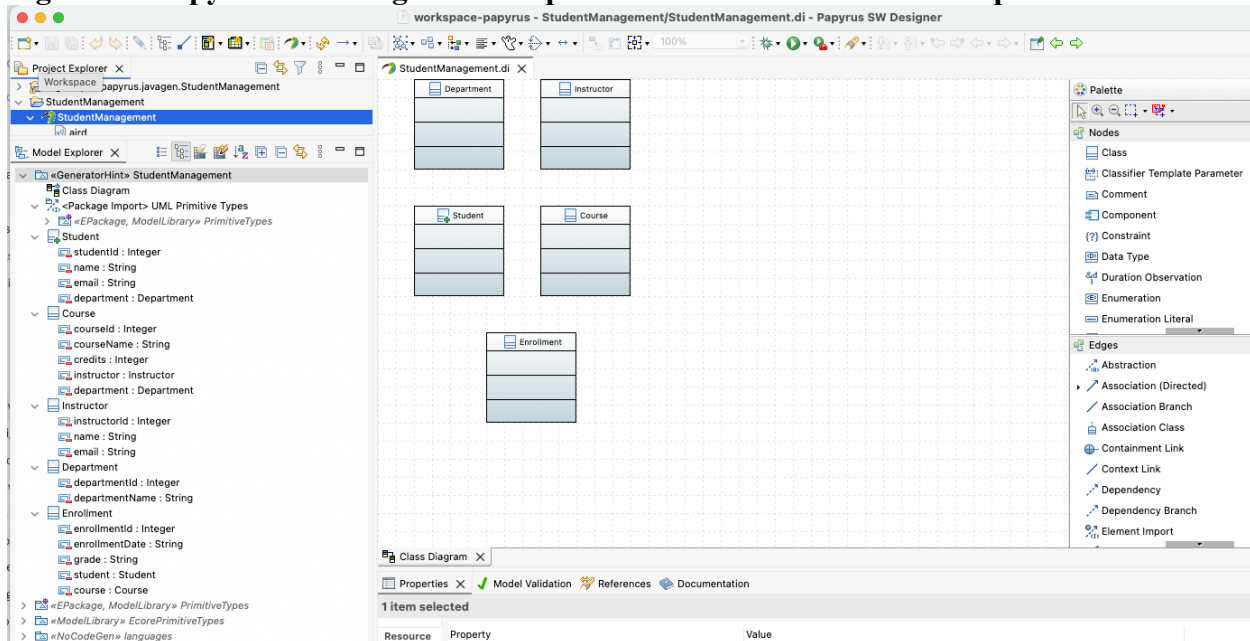


Figure 42. Designer > Generate Code menu option in SW Designer version

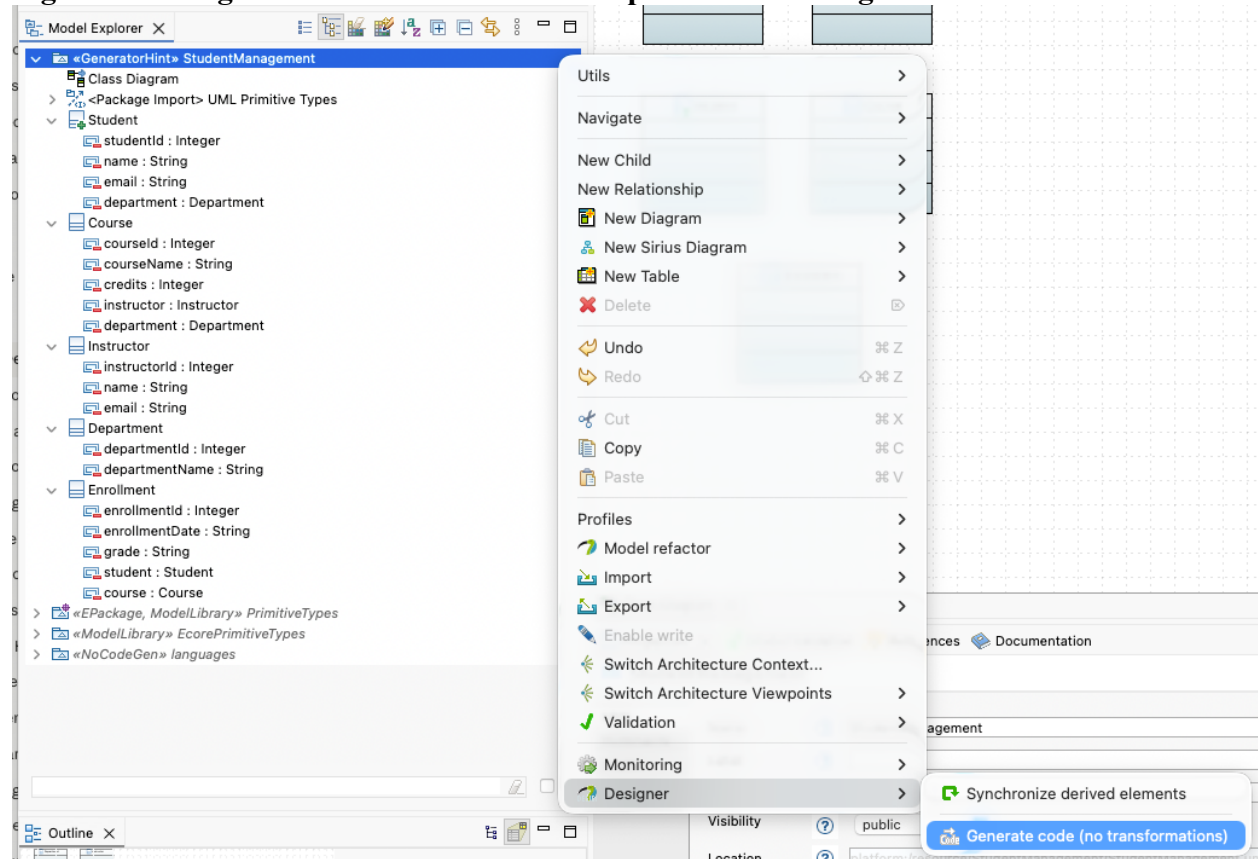


Figure 43. Selection of “Standard Java”

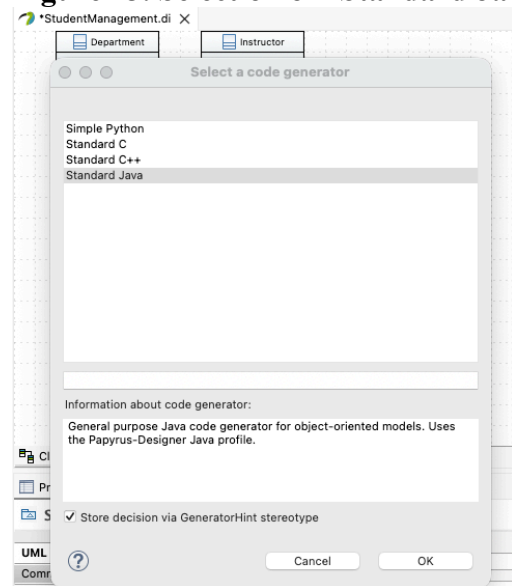


Figure 44. Creation of target project acceptance modal

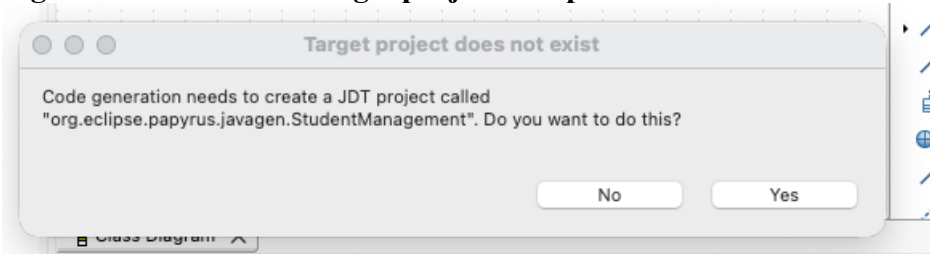


Figure 45. Confirmation of file creation

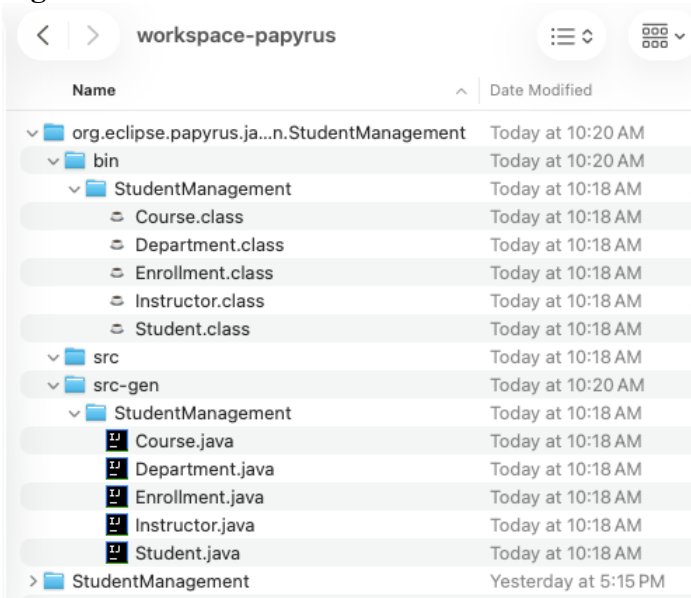


Figure 46. Warning modal upon loading new project in IntelliJ IDEA

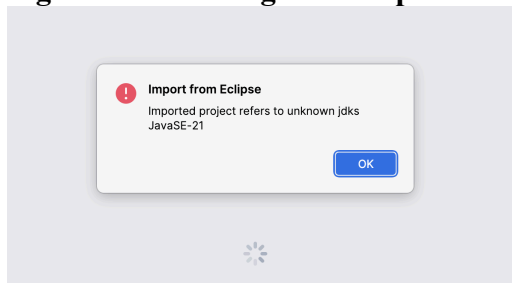


Figure 47. Checking JDK settings in IntelliJ IDEA (updated to SDK 21)

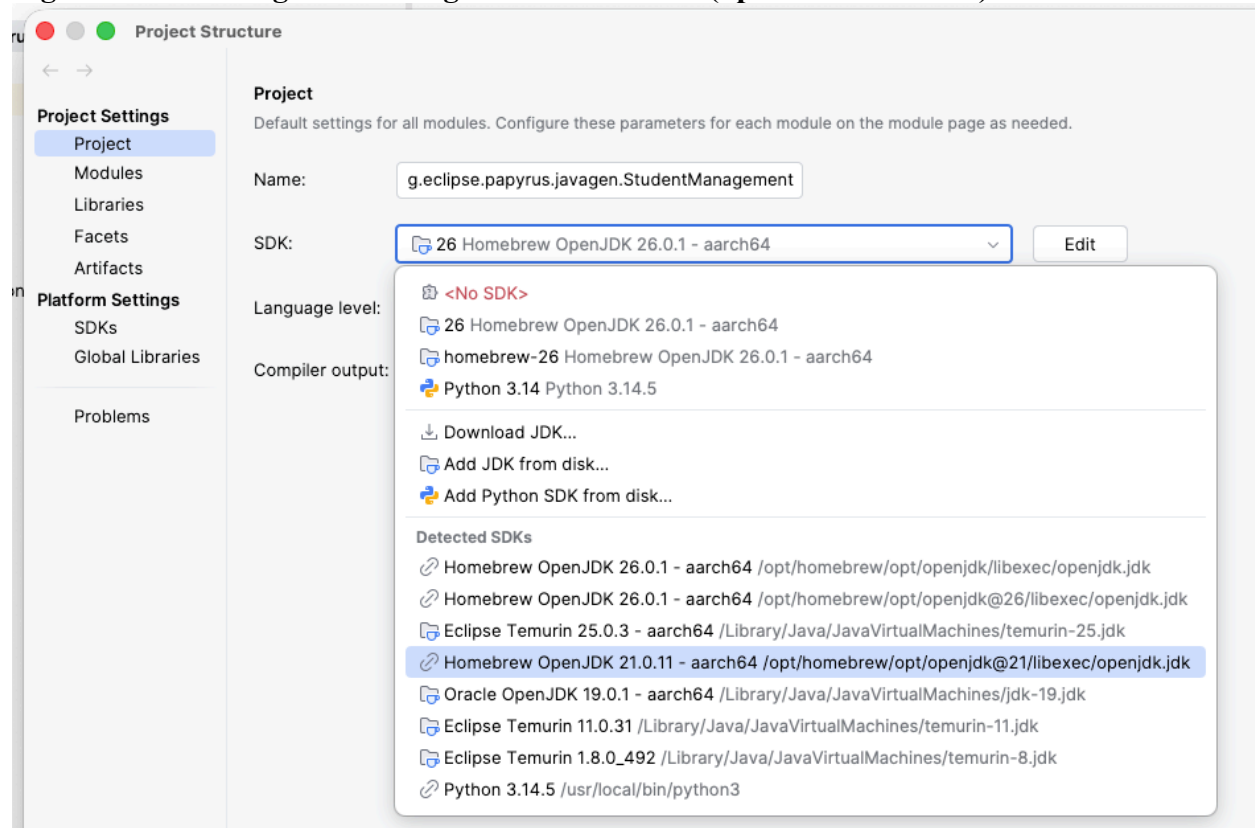


Figure 48. Confirming the files in IntelliJ IDEA

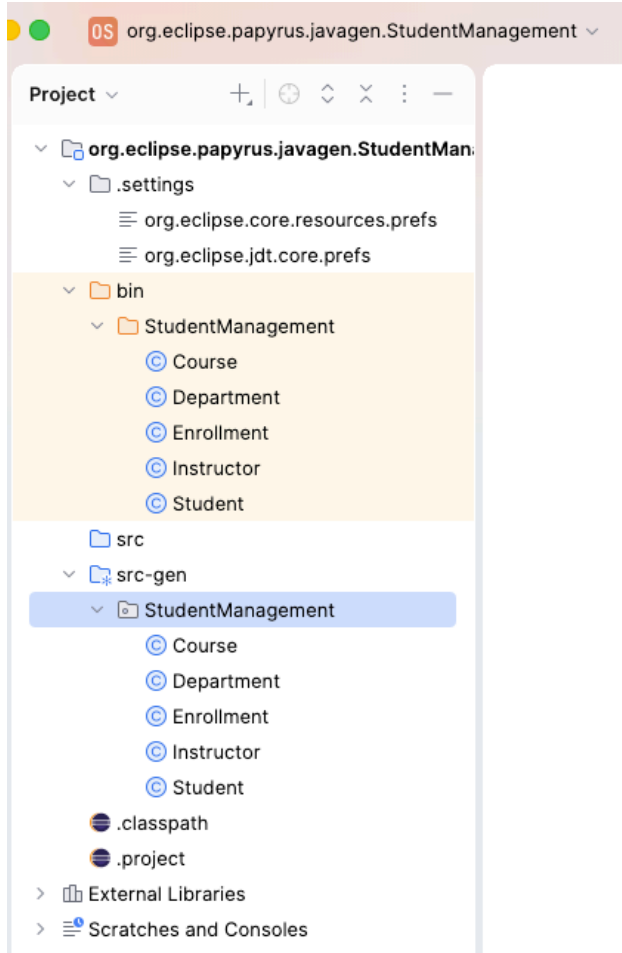


Figure 49. Running Build

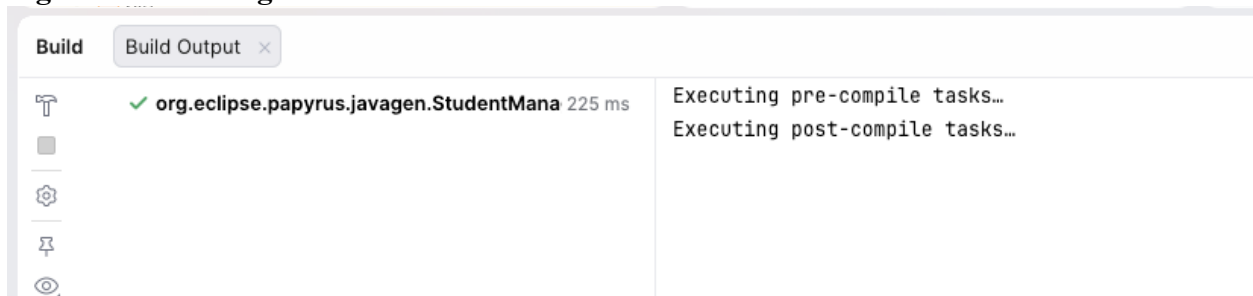


Figure 50. Student Class Validation – src-gen v instructions

Generated source files should not be edited. The changes will be lost when...

1

>

...

4

5

package StudentManagement;

6

7

8

>

11

public class Student { 1 usage

12

13

14

15

private int studentId; no usages

16

17

18

19

private String name; no usages

20

21

22

23

private String email; no usages

24

25

26

27

private Department department; no usages

28

}

Student.java

package StudentManagement;

public class Student {

private int studentId;

private String name;

private String email;

private Department department;

}

Figure 51. Course Class Validation – src-gen v instructions

// -----
// Code generated by Papyrus Java
// -----
5
package StudentManagement;
6
7
8
9
10
11
public class Course { 1 usage
12
13
14
15
private int courseId; no usages
16
17
18
19
private String courseName; no usages
20
21
22
23
private int credits; no usages
24
25
26
27
private Instructor instructor; no usages
28
29
30
31
private Department department; no usages
32
}

Course.java

package StudentManagement;

public class Course {

private int courseId;

private String courseName;

private int credits;

private Instructor instructor;

private Department department;

}

Figure 52. Instructor class validation

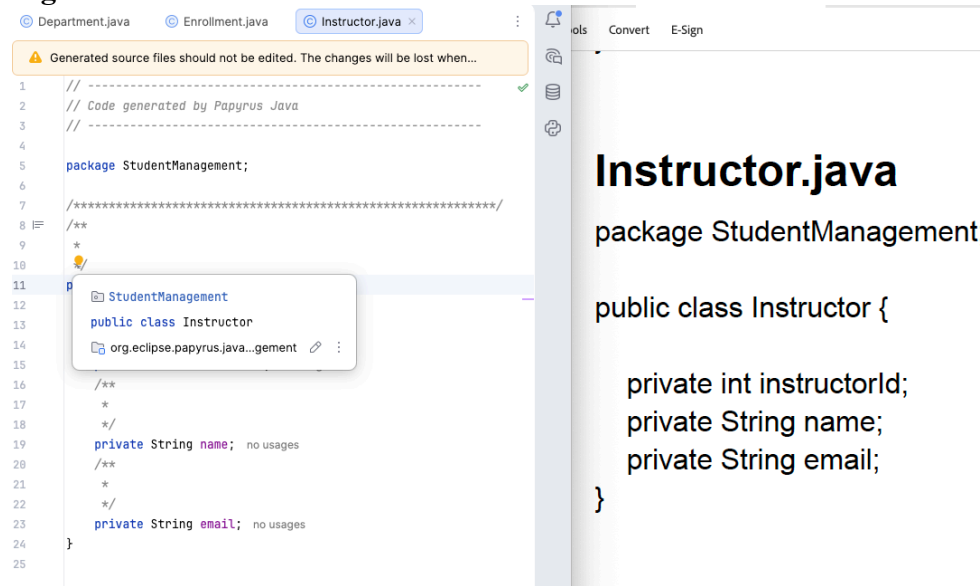


Figure 53. Department class validation

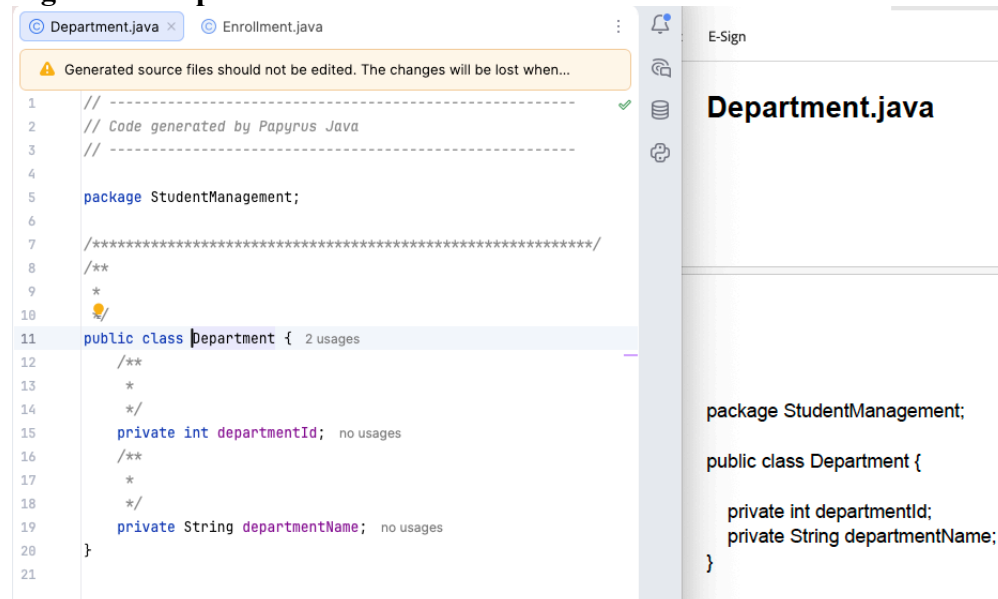


Figure 54. Enrollment class validation

Enrollment.java

Generated source files should not be edited. The changes will be lost when...

```
1 // -----
2 // Code generated by Papyrus Java
3 // -----
4
5 package StudentManagement;
6
7 /**
8  *
9  */
10
11 public class Enrollment { no usages
12     /**
13      *
14      */
15     private int enrollmentId; no usages
16     /**
17      *
18      */
19     private String enrollmentDate; no usages
20     /**
21      *
22      */
23     private String grade; no usages
24     /**
25      *
26      */
27     private Student student; no usages
28     /**
29      *
30      */
31     private Course course; no usages
32 }
```

E-Sign

```
package StudentManagement;

public class Department {

    private int departmentId;
    private String departmentName
}
```

Enrollment.java

```
package StudentManagement;

public class Enrollment {

    private int enrollmentId;
    private String enrollmentDate;
    private String grade;
    private Student student;
    private Course course;
}
```

These minimal classes are sufficient properties to Java classes and fields.

Constructors, getters, setters, validation required by the instructor.

34. Compiling a

APPENDIX 5: Test Scenarios Table

Test scenarios listed here for reference and to reduce lab report density:

Test Case ID	Test Item	Test Objective	Input / Preconditions	Execution Steps	Expected Result
TC-01	Student.java	Verify completeness and correct typing of all Student fields.	Generated Student.java exists and compiles.	Inspect field declarations in Student.java.	Fields studentId:int, name:String, email:String, department:Department are present with correct Java types.
TC-02	Student–Department relationship	Validate correct reference storage for Student.department.	Instantiate Student and Department objects.	Assign student.setDepartment(dept) and retrieve via getter.	Getter returns the same Department instance; reference integrity preserved.
TC-03	Course–Instructor relationship	Validate correct reference storage for Course.instructor.	Instantiate Course and Instructor objects.	Assign course.setInstructor(instr) and retrieve via getter.	Getter returns the same Instructor instance; reference integrity preserved.
TC-04	Course–Department relationship	Validate correct association between Course and Department.	Instantiate Course and Department objects.	Assign course.setDepartment(dept) and retrieve via getter.	Getter returns assigned Department; multiple Courses may reference same Department.
TC-05	Enrollment linking behavior	Verify Enrollment stores Student, Course, and required attributes.	Instantiate Enrollment, Student, and Course.	Assign student, course, enrollmentDate, and grade fields.	Enrollment contains correct references and non-null date/grade values.
TC-06	Generated project (all classes)	Confirm compilation and absence of unresolved types or undef.	Generated project opened in IntelliJ IDEA.	Trigger full rebuild.	Build completes with zero errors; no undef or unresolved type regressions.

REFERENCES:

- Eclipse Foundation. (n.d.). Papyrus: Eclipse Papyrus modeling environment. Eclipse.
<https://eclipse.dev/papyrus/>
- Eclipse Foundation. (n.d.). Papyrus downloads. Eclipse. Retrieved July 17, 2026, from
<https://eclipse.dev/papyrus/download.html>
- Eclipse Foundation. (n.d.). Papyrus documentation. Eclipse.
<https://eclipse.dev/papyrus/documentation.html>
- Eclipse Foundation. (n.d.). Papyrus relatives. Eclipse. <https://eclipse.dev/papyrus/relatives.html>
- Radermacher, A. (n.d.). Papyrus Software Designer. Eclipse Marketplace. Retrieved July 18, 2026, from <https://marketplace.eclipse.org/content/papyrus-software-designer>
- Eclipse Foundation. (n.d.). How to add an update site (Papyrus Software Designer help). Eclipse Marketplace. <https://marketplace.eclipse.org/content/papyrus-software-designer/help>
- Eclipse Foundation. (n.d.). Papyrus Software Designer 3.2.0 update site [Software repository]. Eclipse Downloads. Retrieved July 17, 2026, from
<https://download.eclipse.org/modeling/mdt/papyrus/papyrus-desktop/components/designer/releases/3.2.0/p2/>
- Eclipse Foundation. (n.d.). Eclipse downloads packages. Eclipse. Retrieved July 17, 2026, from
<https://www.eclipse.org/downloads/packages/>
- Eclipse Foundation. (n.d.). Eclipse Papyrus for Real Time (Papyrus-RT) [Project page]. Eclipse Foundation Projects. <https://projects.eclipse.org/projects/modeling.papyrus-rt>
- Eclipse Foundation. (2017, August 23). First release of Eclipse Papyrus for Real Time: Modeling tool for UML-RT [Press release]. Eclipse Newsroom.
<https://newsroom.eclipse.org/news/announcements/first-release-eclipse-papyrus-real-time-modeling-tool-uml-rt>
- SaaSHub. (n.d.). *Modelio alternatives*. Retrieved July 17, 2026, from
<https://www.saashub.com/modelio-alternatives>
- Visual Paradigm. (n.d.). *Visual Paradigm Community Edition download*. Retrieved July 17, 2026, from <https://www.visual-paradigm.com/download/community.jsp>

Methodology and Use of AI Tools

I relied on the coursework “Setup and Code” guide, Class Connect and other Canvas & Coursera materials for CSYE6255.

The analysis, observations, and conclusions reflect the author's own hands-on experience with the tool and tasks analyzed. All content has been reviewed for accuracy and represents the author's understanding of the results.

Reference to Claude LLM Search for MARTE links and options to generate Java from UML design: <https://claude.ai/share/6d74f648-30c2-446f-8ff7-49579775cfda>

Reference to Claude LLM conversation on draft and rubric compliance:
<https://claude.ai/share/f960b97b-a2bb-414f-9f0a-1d1815844d76>

Reference to CoPilot formatting Appendix 5 test cases into an IEEE-style test matrix:
<https://copilot.microsoft.com/shares/KhAcR8xfa4yhHcf17HGao>